

Fabric-as-Code

A Terraform North Star for repeatable Microsoft Fabric platforms

Capacity | Workspaces | Security | Git | Items | Workflows | Monitoring

Understand the control planes

Microsoft Fabric spans distinct lifecycle boundaries. A complete implementation uses the right tool for each one:

Layer	Manages	Tooling here
Azure	Fabric capacity and runner network	<code>hashicorp/azurerm</code> + AzAPI
Fabric	Workspaces, roles, Git, and items	<code>microsoft/fabric</code> provider
Data	Warehouse schemas, tables, procedures	Entra-authenticated T-SQL
Gaps	CMK and inbound policy lifecycle	Public Fabric REST bridges

Terraform coordinates the graph; it does not pretend every operation belongs to the same API.

What this reference covers

Area	Delivered pattern
Platform	F-SKU capacity plus separate authoring and CI/CD workspaces
Governance	Roles, optional CMK, inbound policy, and monitoring
Source	One canonical <code>fabric-git/</code> definition tree
Delivery	Environment tfvars, PR validation, deploy-on-merge
Safety	Private state, OIDC, delete rejection, convergence checks
Enablement	End-to-end guides plus agents, skills, prompts, and MCP

Explicit handoffs remain explicit: tenant settings, repository creation, the Fabric Platform CMK enterprise app, and portal-only Workspace monitoring.

End-to-end deployment flow

1. **Load configuration** from `terraform/environments/<environment>.tfvars` .
2. **Provision or adopt capacity** and resolve its Fabric object GUID.
3. **Create or import two workspaces** with distinct lifecycle owners.
4. **Apply workspace settings** before Git or item operations.
5. **Connect Git** only to the authoring workspace.
6. **Deploy Fabric items** only to the CI/CD workspace.
7. **Apply Warehouse SQL**, verify paths, and require zero-change convergence.

The guarded PowerShell wrapper runs format, init, validate, tests, plan, apply, and convergence in that order.

Browse from docs to code

Guide	Owning implementation
01-capacity.md	modules/capacity
02-workspaces.md	modules/workspace
03-workspace-settings.md	Root roles, CMK, firewall, monitoring
04-git-integration.md	fabric_workspace_git + caller credentials
05-content-files.md	modules/items , modules/sql , fabric-git/

Every provider resource includes a nearby **Learn more** link to its official Terraform Registry documentation.

Keep ownership unambiguous

Workspace	Daily purpose	Definition owner
Git authoring	Update from Git, author, commit to Git	Fabric Git + repository
CI/CD deployment	Validate merged, repeatable output	Terraform

- Git synchronization targets only the authoring workspace.
- Terraform item resources target only the CI/CD workspace.
- Both consume the same committed `fabric-git/` definitions.
- Production-style CI/CD workspaces are not edited in place.

This avoids two systems reconciling the same item in the same workspace.

Choose the item profile deliberately

Profile	Items	Use when
p0	Lakehouse, Warehouse, Notebook	CMK-compatible governed core
all	P0 + Environment, Eventhouse, KQL DB, Variable Library, ML Experiment, Pipeline	Complete demonstration

Power BI Report, Semantic Model, and Eventstream are explicitly outside P0.
The repo does not claim out-of-scope items as delivered.

Environment files are public templates until adopters replace placeholders and remove the `# deployment: template` marker.

Workspace governance is part of the graph

Control	Implementation	Important boundary
Roles	Native <code>fabric_workspace_role_assignment</code>	Customer principal object IDs
CMK	<code>terraform_data</code> + GA encryption API	<code>p0</code> , tenant setting, RSA key, permissions
Inbound policy	<code>terraform_data</code> + communication policy API	Tenant controls + workspace eligibility
Monitoring	Firewall diagnostics + authenticated canaries	VNet runner and private backend

Git and item resources depend on applicable settings, so deployment does not race ahead of workspace policy.

CMK: reusable, but never casual

Before assignment:

1. Enable **Apply customer-managed keys** in the Fabric tenant.
2. Instantiate the documented Fabric Platform CMK enterprise application.
3. Create an RSA Key Vault or Managed HSM key with soft delete and purge protection.
4. Grant get, wrap, and unwrap permissions.
5. Use a **versionless** key URI and a supported item profile.

Terraform then assigns, rotates, resets, and polls status. Removing a map entry does not silently reset encryption.

One source tree, portable references

- `fabric-git/` contains Fabric-native Lakehouse, Notebook, and Pipeline files.
- `.platform` logical IDs remain stable source identity.
- Terraform reads logical IDs but uploads only supported definition parts.
- `TextReplace` resolves target Notebook and workspace IDs **in memory**.
- No rendered tenant-specific copies are written back to Git.
- `fabric-git/sql/` is the canonical source for Warehouse T-SQL.

The same definitions can be synchronized into authoring and deployed into multiple target workspaces.

Git integration: connect, do not co-own

1. Create the GitHub or Azure DevOps repository outside this stack.
2. Create and share a Fabric Git cloud connection.
3. Store only the non-secret connection ID in tfvars.
4. Select caller-specific credentials before provider refresh.
5. Connect the authoring workspace with `PreferRemote`.

The PAT remains in Fabric. CI receives neither the PAT nor repository secrets. Repository creation and branch protection remain organization-owned prerequisites.

Environment tfvars drive promotion

pull request

- > select added/changed tfvars
- > backend-free format + validate + mocked tests
- > strict tfvars contract

merge to main

- > select the same non-template environment
- > matching GitHub Environment + OIDC
- > guarded plan/apply/convergence

PRs never receive privileged deployment credentials.

Private runner and state boundary

- A GitHub-hosted larger runner is injected into a delegated Azure subnet.
- A deny-inbound NSG protects the ephemeral runner NIC.
- `0.0.0.0/0` routes through Azure Firewall for controlled egress.
- `PowerBI` and `Sql` service tags permit Warehouse TDS on TCP 1433.
- Terraform state uses a private endpoint, Entra authentication, locking, versioning, and soft deletion.

The unprivileged selector runs on `ubuntu-latest` ; only actual deployments require the private runner.

Monitoring where stable contracts exist

- Azure Firewall `allLogs` and `AllMetrics` flow to Log Analytics.
- Every deployment can verify private state DNS, service tags, firewall rules, Fabric REST, optional CMK/inbound policy, and Warehouse TDS.
- A six-hour canary is opt-in after the VNet runner is configured.
- Fabric-native Workspace monitoring remains a documented portal handoff until a supported lifecycle API is available.

The docs distinguish implemented automation from customer prerequisites and manual product boundaries.

GitHub Copilot tooling enables adoption

Tool	Purpose
Project instructions	Preserve architecture, ownership, and safety boundaries
Terraform instructions	Apply repository-specific IaC conventions
Deployment reviewer agent	Produce read-only scope and readiness evidence
Validation skill + prompt	Run the repeatable release gate
Azure + GitHub MCP	Read-only context from official tool endpoints

The tooling points back to `DELIVERY_SCOPE.md` , the end-to-end guides, and official provider documentation.

Validation is executable evidence

- `terraform fmt -check -recursive`
- `clean terraform init -backend=false + terraform validate`
- **19** mocked Terraform tests across both stacks
- tfvars contracts + new-environment selection regression
- PowerShell, JSON, and documentation-link parsing
- public repository hygiene and secret scanning
- guarded apply plus zero-change convergence

A live publisher environment is evidence, not a prerequisite for adopters.

Adopt the pattern

```
git clone https://github.com/melabadi/fabric-as-code
Set-Location fabric-as-code
Copy-Item terraform/environments/p0.tfvars.example `
  terraform/environments/<environment>.tfvars
```

1. Replace placeholders and remove `# deployment: template`.
2. Configure a matching GitHub Environment and federated identity.
3. Bootstrap the private backend and eligible VNet runner.
4. Open a PR, review the contract, then deploy after merge.

Start with `docs/deployment/README.md` and `docs/DELIVERY_SCOPE.md`.